

degrees. When you create your own shape-drawing functions, stick to the relative turtle movement functions like `turtle.forward()`, `turtle.left()`, and `turtle.right()` instead of absolute turtle movement functions like `turtle.goto()`.

Next, the `depth` argument tells the function whether it should draw a white square or a gray one:

Python

```
# Alternate between white and gray (with black border):
if depth % 2 == 0:
    turtle.pencolor('black')
    turtle.fillcolor('white')
else:
    turtle.pencolor('black')
    turtle.fillcolor('gray')
```

If `depth` is even, the `depth % 2 == 0` condition is `True`, and the square's *fill color* is white. Otherwise, the code sets the fill color to gray. Either way, the border of the square, determined by the *pen color*, is set to black. To change either of these colors, use strings of common color names, like `red` or `yellow`, or an HTML color code comprising a hash mark and six hexadecimal digits, like `#24FF24` for lime green or `#AD7100` for brown.

The website <https://html-color.codes> has charts for many HTML color codes. The fractals in this black-and-white book lack color, but your computer can render your own fractals in a bright range of colors!

With the colors set, we can finally draw the four lines of the actual square:

Python

```
# Draw a square:
turtle.begin_fill()
for i in range(4): # Draw four lines.
    turtle.forward(size)
    turtle.right(90)
turtle.end_fill()
```

To tell the `turtle` module that we intend to draw a filled-in shape and not just the outline, we call the `turtle.begin_fill()` function. Next is a `for` loop that draws a line of length `size` and turns the turtle 90 degrees to the right. The `for`